

FRONTEND DESIGN

UNIT I: Core Web Foundations

Semantic Structure and Layout Design: HTML5 semantic elements, forms, accessibility basics, layout design using Flexbox and CSS Grid, structuring clean and maintainable web pages.

Responsive Design and Modern Styling: Responsive design using media queries (mobile-first), introduction to modern styling tools (Tailwind CSS via CDN), basics of web concepts including browsers, HTTP/HTTPS, and DNS.

UNIT II: JavaScript and Dynamic UI Development

JavaScript and DOM Manipulation: JavaScript fundamentals, events, DOM selection and updates, form validation, interactive UI behaviour.

Asynchronous Data Handling: Fetch API for API integration, JSON handling, loading states, error handling in UI, introduction to breaking problems into smaller steps.

UNIT III: Modern Frontend Development

Component-Based Development: Introduction to React (via CDN), components, props, basic state (`useState`), building reusable UI components.

Modern UI and Resource Usage: Using Tailwind CSS for rapid UI design, basic UI principles and consistency, using CDN and simple package tools for integrating libraries.

UNIT IV: Production, AI-Assisted Development and Deployment

AI-Assisted Development and Debugging: Using AI tools (ChatGPT, Gemini, Claude, Copilot) for code generation, writing minimal prompts, refining output, debugging generated code.

Deployment and Real-World Practices: Version control using Git basics, hosting frontend applications using Vercel and Netlify, basic performance optimization, debugging in production.

Textbooks

1. Duckett, Jon, *HTML & CSS: Design and Build Websites*, Wiley, 2011.
2. Robbins, Jennifer Niederst, *Learning Web Design: A Beginner's Guide to HTML, CSS, JavaScript, and Web Graphics*, O'Reilly Media, 2018.
3. Haverbeke, Marijn, *Eloquent JavaScript: A Modern Introduction to Programming*, 3rd Edition, No Starch Press, 2018.

FRONTEND DESIGN LAB

Note: From 6th Question onwards usage of AI can be encouraged for faster development

1. Create a Structured Webpage using Semantic HTML:

- a. Design a webpage using `<header>`, `<nav>`, `<section>`, `<article>`, `<footer>` for a college event page.
- b. Include a schedule table, speaker list, images, and navigation links.

2. Design a Responsive Layout using CSS (Flexbox & Grid):

- a. Create a webpage layout with header, sidebar, content, and footer using Flexbox and CSS Grid.
- b. Apply box model, spacing, and alignment properly.

3. Create a Mobile-First Responsive Webpage:

- a. Design a product landing page using media queries (mobile-first approach).
- b. Adjust layout, font sizes, and visibility for mobile, tablet, and desktop.

4. Build a Styled UI using Tailwind CSS (CDN):

- a. Use Tailwind CSS CDN to design a responsive card layout.
- b. Create reusable components like buttons, cards, and navbar.

5. Build a Registration Form with Validation:

- a. Create a form with input types: text, email, password, radio, checkbox, select.
- b. Validate using JavaScript (pattern, required, length) and show dynamic error messages.

6. Build a Calculator using JavaScript and Enhance Features:

- a. Create a calculator for basic operations (+, -, ×, ÷) using JavaScript.
- b. Enhance it with:
 - Keyboard input support
 - Error handling (divide by zero)
 - Clear/reset functionality

7. Implement DOM Manipulation for a Dynamic UI:

- a. Create a task list where users can add, edit, and delete items using `querySelector`.
- b. Dynamically update styles and content on user actions.

8. Fetch and Display API Data with Error Handling:

- a. Fetch data from a public API (e.g., users/posts) using Fetch API.
- b. Display data in a table with:

- Loading indicator
- Error message if API fails

9. Convert Static Page into React Components (CDN):

- a. Take an existing HTML page and convert it into React components using CDN.
- b. Use props and basic state (useState) for dynamic data.

10. Build a Mini Dashboard using React:

- a. Create a dashboard with components (navbar, sidebar, content cards).
- b. Display dynamic data (API or static JSON) and manage state.

11. Solve UI Problem using AI with Minimal Prompts:

- a. Given a broken UI (layout or styling issue), fix it using AI assistance.
- b. Reduce prompt size step-by-step and document improvements.

12. Deploy a Production-Ready Frontend Application:

- a. Build a complete frontend app (React + API integration).
- b. Use Git for version control and deploy using Vercel or Netlify.

Submit:

- Live deployed link
- Code repository
- Prompts used and improvements made